

PYLOOM QUESTION SET

(Easy: Core Math & Logic | Medium: DSA — Intermediate Level | Hard: Graph & Pie Chart Output)

EASY LEVEL

Question 1: Voting Eligibility Checker (Numbers & Booleans)

Problem Statement: An election portal checks whether a person is eligible to vote. A person is eligible only if their age is 18 or above.

Processing Steps

1. Input Node: Take the person's age.
2. Eligibility Check Node: Compare the age against the voting age limit (18).

Input Data: 20 (Age)

Expected Output: True (Eligible to Vote)

Block Flow Diagram

[1. Input Node] → [2. Greater/Equal Node (≥ 18)] → [3. Output Node]

Question 2: BMI Calculator (Numbers)

Problem Statement: A fitness app calculates a person's Body Mass Index (BMI) using the formula: $BMI = \text{Weight (kg)} / \text{Height}^2 \text{ (m)}$, rounded to 1 decimal place.

Processing Steps

1. Input Node: Take Weight and Height.
2. Square Node: Square the height value.
3. Divide Node: Divide weight by the squared height.
4. Round Node: Round the result to 1 decimal place.

Input Data: Weight: 70 kg, Height: 1.75 m

Expected Output: 22.9

Block Flow Diagram

[1. Input] → [2. Square Node (Height^2)] → [3. Divide Node (Weight/H^2)] → [4. Round Node] → [5. Output]

Question 3: Palindrome String Checker (Strings)

Problem Statement: Given a word, check if it reads the same backward as forward (a palindrome).

Input Data: "radar"

Expected Output: True

Block Flow Diagram

[1. Input Node] → [2. Reverse String Node] → [4. Equals Node ($==$)] → [5. Output]
[1. Input Node] → [3. Direct String Path] → [4. Equals Node ($==$)] → [5. Output]

Question 4: Fahrenheit to Celsius Converter (Numbers)

Problem Statement: A weather app converts a temperature reading from Fahrenheit to Celsius. Formula:
 $C = (F - 32) \times 5 / 9$.

Processing Steps

1. Input Node: Take the temperature reading in °F.
2. Convert Node: Apply the Fahrenheit-to-Celsius formula.

Input Data: 98.6 (Temperature in °F)

Expected Output: 37.0 (Temperature in °C)

Block Flow Diagram

[1. Input Node] → [2. Convert Node ((F-32)x5/9)] → [3. Output Node]

Question 5: Average of Numbers (Numbers)

Problem Statement: Given a list of numbers, calculate their average (mean) value.

Processing Steps

1. Input Node: Take the list of numbers.
2. Sum Node: Add up all the numbers.
3. Divide Node: Divide the sum by the count of numbers.

Input Data: [10, 20, 30, 40] (List of Numbers)

Expected Output: 25.0

Block Flow Diagram

[1. Input] → [2. Sum Node] → [3. Divide Node (/ Count)] → [4. Output]

MEDIUM LEVEL

Question 1: Binary Search Locator (DSA — Searching Algorithms)

Problem Statement: Given a sorted array and a target value, find the index of the target using Binary Search by repeatedly halving the search range.

Processing Steps

1. Input Node: Take the sorted array and the target value.
2. Midpoint Node: Calculate the middle index of the current search range.
3. Compare Node (Conditional): If the middle value equals the target, stop; if the target is smaller, search the left half; if larger, search the right half.
4. Repeat Node: Repeat the Midpoint and Compare steps on the narrowed range until the target is found.

Input Data: Array: [2, 4, 6, 8, 10, 12, 14], Target: 10

Expected Output: 4 (Index of value 10)

Block Flow Diagram

[1. Input] → [2. Midpoint Node] → [3. Compare Node (==? <? >?)] → [4. Repeat on Half-Range] → [5. Output]

Question 2: Leap Year Checker (DSA — Conditional Logic)

Problem Statement: Given a year, determine whether it is a leap year. A year is a leap year if it is divisible by 4, except for century years (divisible by 100), which must also be divisible by 400 to qualify.

Processing Steps

1. Input Node: Take the year value.
2. Divisible-by-4 Check Node (Conditional): Check if the year is divisible by 4.
3. Century Rule Check Node (Conditional): If the year is also divisible by 100, it must additionally be divisible by 400 to remain a leap year.
4. Output Node: Return the final leap year result.

Input Data: 2000

Expected Output: True (Divisible by 4, by 100, and by 400)

Block Flow Diagram

[1. Input] → [2. Divisible-by-4 Check] → [3. Century Rule Check (÷100? → ÷400?)] → [4. Output]

Question 3: Sorting a List of Numbers (DSA — Sorting Algorithms)

Problem Statement: Given an unsorted list of numbers, sort them in ascending order using Bubble Sort — repeatedly compare adjacent elements and swap them if they are in the wrong order.

Processing Steps

1. Input Node: Take the unsorted list of numbers.
2. Compare & Swap Node (Conditional): Compare each pair of adjacent elements; swap them if the left element is greater than the right.
3. Pass Repeat Node: Repeat the comparison pass over the list until no more swaps are needed.
4. Output Node: Return the fully sorted list.

Input Data: [64, 25, 12, 22, 11]

Expected Output: [11, 12, 22, 25, 64]

Block Flow Diagram

[1. Input] → [2. Compare & Swap Node (left>right?)] → [3. Pass Repeat Node] → [4. Output]

HARD LEVEL (Graph & Pie Chart Output Questions)

Question 1: Hard Level — Monthly Expense Pie Chart (Percentage Calculation & Pie Chart Output) (4-Node Flow)

Problem Statement: A personal finance app records monthly expenses across 4 categories: Rent, Food, Transport, and Entertainment. Calculate each category's percentage share of the total spending, then render the result as a pie chart.

Processing Steps

1. Input Node: Take the expense amounts for each category.
2. Total Sum Node: Add up all the category amounts.
3. Percentage Calculator Node: Divide each category amount by the total and multiply by 100.
4. Pie Chart Render Node: Plot each category's percentage share as a pie chart slice.

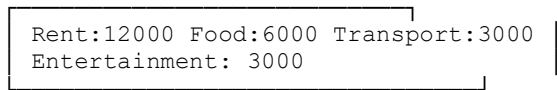
Input Data: Rent: 12000, Food: 6000, Transport: 3000, Entertainment: 3000 (Total: 24000)

Expected Data Output:

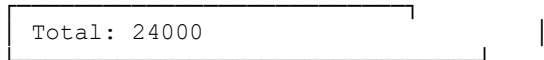
Rent: 50% | Food: 25% | Transport: 12.5% | Entertainment: 12.5%

PyLoom Flow Node Diagram (Canvas Layout)

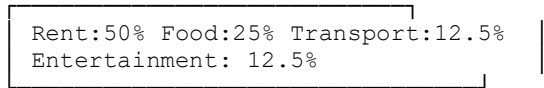
[1. INPUT NODE]



[2. TOTAL SUM NODE]

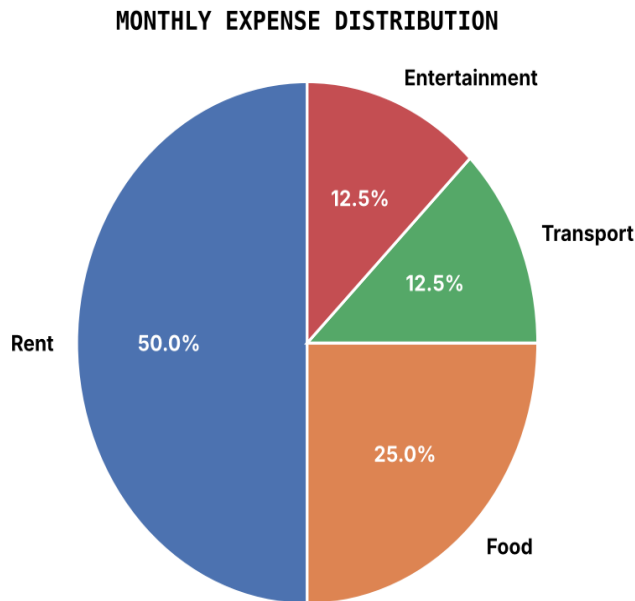


[3. PERCENTAGE CALCULATOR NODE]



[4. PIE CHART RENDER NODE]

Visual Render Output Display — Monthly Expense Pie Chart



STATUS: PASS | PERCENTAGE CHECK: VALID | CREDITS: +50

Question 2: Hard Level — Weekly Sales Bar Chart (Target Comparison & Bar Chart Output) (3-Node Flow)

Problem Statement: A store tracks daily sales for 5 days against a fixed sales target of \$200. Mark each day as “Above Target” or “Below Target”, then render the results as a bar chart.

Processing Steps

- 1. Input Node: Take the daily sales list and the target value.
- 2. Target Comparison Node (Conditional): For each day, check if sales $\geq$ target.

3. Bar Chart Render Node: Plot each day's sales as a bar, coloring it green (Above Target) or red (Below Target).

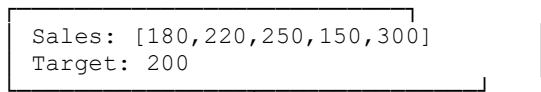
Input Data: Daily Sales: [180, 220, 250, 150, 300] | Target: 200

Expected Data Output:

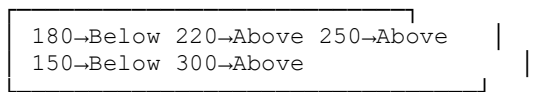
Mon: Below Target | Tue: Above Target | Wed: Above Target | Thu: Below Target | Fri: Above Target

PyLoom Flow Node Diagram (Canvas Layout)

[1. INPUT NODE]

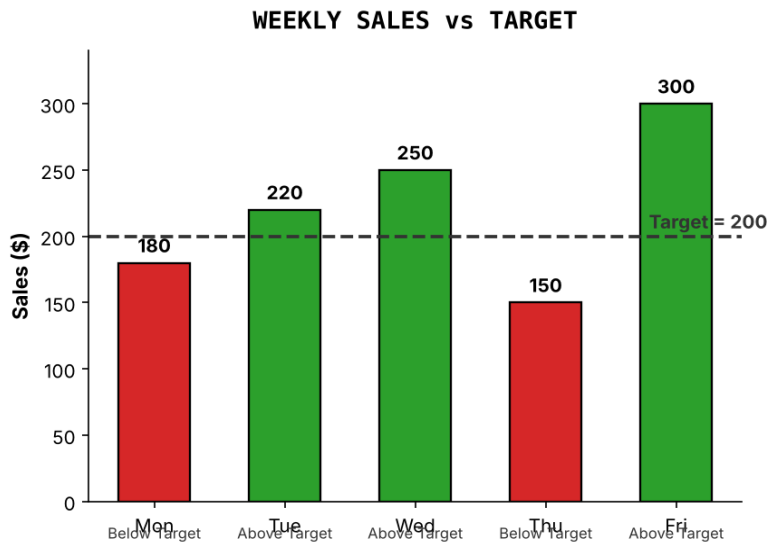


[2. TARGET COMPARISON NODE (Sales ≥ 200?)]



[3. BAR CHART RENDER NODE]

Visual Render Output Display — Weekly Sales Bar Chart



STATUS: PASS | TARGET COMPARISON: VALID | CREDITS: +50

Question 3: Hard Level — Student Grade Distribution Pie Chart (Classification, Counting & Pie Chart Output) (4-Node Flow)

Problem Statement: Given a list of student scores, classify each score into a grade (A: ≥90, B: 75–89, C: 50–74, F: <50), count how many students fall into each grade, and render the distribution as a pie chart.

Processing Steps

1. Input Node: Take the list of student scores.
2. Grade Classifier Node (Conditional): Assign a grade to each score.
3. Count Aggregator Node: Count the number of students in each grade.

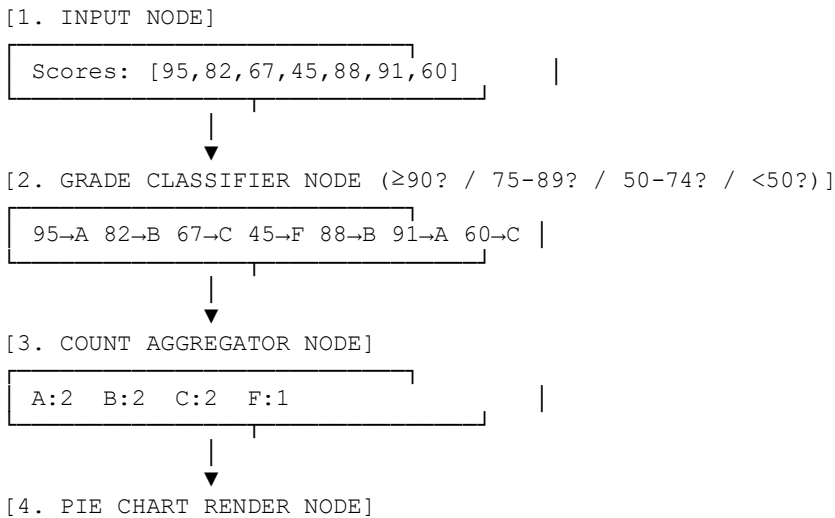
4. Pie Chart Render Node: Plot the grade counts as pie chart slices.

Input Data: [95, 82, 67, 45, 88, 91, 60]

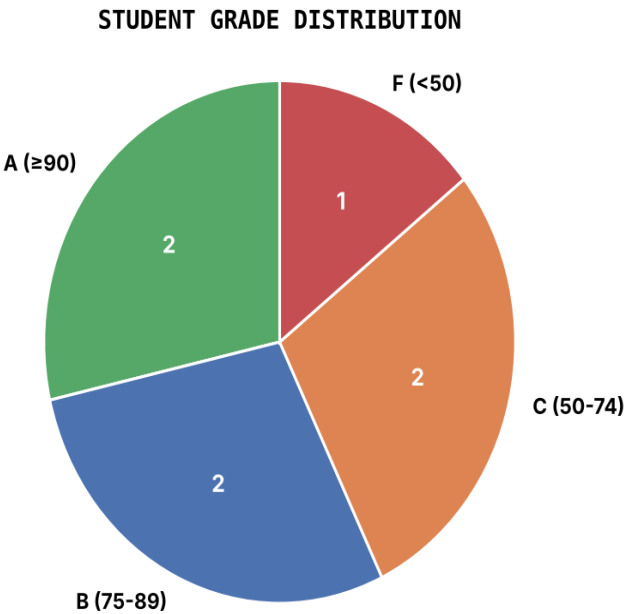
Expected Data Output:

A: 2 | B: 2 | C: 2 | F: 1

PyLoom Flow Node Diagram (Canvas Layout)



Visual Render Output Display — Student Grade Distribution Pie Chart



STATUS: PASS | CLASSIFICATION CHECK: VALID | CREDITS: +50